

Due: Wednesday, April 17 at 11:59 pm

Deliverables:

1. Submit your predictions for the test sets to Kaggle as early as possible. Include your Kaggle scores in your write-up (see below). The Kaggle competition for this assignment can be found at

- <https://www.kaggle.com/competitions/cs189-hw6-cifar-spring-2024>

2. The written portion:

- Submit a PDF of your homework, **with an appendix listing all your code**, to the Gradescope assignment titled “Homework 6 Write-Up”. Please see section 3.5 for an easy way to gather all your code for the submission (you are **not** required to use it, but we strongly recommend it).
- In addition, please include, as your solutions to each coding problem, the specific subset of code relevant to that part of the problem. Whenever we say “include code”, that means you can either include a screenshot of your code, or typeset your code in your submission (using markdown or $\text{\LaTeX}$).
- You may typeset your homework in LaTeX or Word (submit PDF format, **not** .doc/.docx format) or submit neatly handwritten and scanned solutions. **Please start each question on a new page.**
- If there are graphs, include those graphs in the correct sections. **Do not** put them in an appendix. We need each solution to be self-contained on pages of its own.
- In your write-up, please state with whom you worked on the homework.
- In your write-up, please copy the following statement and sign your signature next to it. (Mac Preview and FoxIt PDF Reader, among others, have tools to let you sign a PDF file.) We want to make it *extra* clear so that no one inadvertently cheats.

“I certify that all solutions are entirely in my own words and that I have not looked at another student’s solutions. I have given credit to all external sources I consulted.”

3. The coding portion:

- Submit all the code needed to reproduce your results to the Gradescope assignment entitled “Homework 6 Code”. Yes, you must submit your code twice: in your PDF write-up following the directions as described above so the readers can easily read it, and once in the format described below so the assignment autograder can parse them correctly.
- Your submission should include all the files in the [neural networks package](#). Simply drag and drop these files directly into the Gradescope submission box; don’t package them in a directory or zip them in any way.
- Do **NOT** submit any data files that we provided.
- Please also include a short file named README listing your name, student ID, and instructions on how to reproduce your results.

- Please take care that your code doesn't take up inordinate amounts of time or memory. If your code cannot be executed by the autograder, your solution cannot be verified.
- Finally, also include any code you write for the [PyTorch](#) section of this homework into your Gradescope submission. If you choose to work directly in Google Colab, feel free to simply download your python notebook and attach it with the rest of your submission.
- At the end, your Gradescope submission screen might look like the following:

Submit Programming Assignment

 Upload all files for your submission

Submission Method

☒  Upload ☐  GitHub ☐  Bitbucket

Add files via Drag & Drop or [Browse Files](#).

Name	Size	Progress	x
__init__.py	0 b		x
activations.py	5.8 KB		x
datasets.py	4.2 KB		x
layers.py	17.3 KB		x
logs.py	3.1 KB		x
losses.py	1.9 KB		x
models.py	9.6 KB		x
optimizers.py	2 KB		x
schedulers.py	1.3 KB		x
utils.py	2.9 KB		x
weights.py	5.4 KB		x
CS189_HW6_NN.ipynb	28.5 KB		x
README.md	0 b		x
train_ffnn.py	3.2 KB		x

1 Honor Code

Declare and sign the following statement:

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

Signature : _____

While discussions are encouraged, *everything* in your solution must be your (and only your) creation. Furthermore, all external material (i.e., *anything* outside lectures and assigned readings, including figures and pictures) should be cited properly. We wish to remind you that consequences of academic misconduct are *particularly severe*!

2 Background

This section will provide a background on neural networks that is designed to help you complete the assignment. There are no questions in this part. The questions for the homework begin in Section 4.

2.1 Neural Networks

Many of the most exciting recent breakthroughs in machine learning have come from “deep” (many-layered) neural networks, such as the [deep reinforcement learning algorithm](#) that learned to play Atari from pixels, or the [ChatGPT](#) model, which generates text that is nearly indistinguishable from human-generated text.

Neural network libraries such as Tensorflow and PyTorch have made training complicated neural network architectures very easy. However, we want to emphasize that neural networks begin with fundamentally simple models that are just a few steps removed from basic logistic regression. In this assignment, you will build two fundamental types of neural network models, all in plain numpy: a **feed-forward fully-connected network**, and a **convolutional neural network**. We will start with the essential elements and then build up in complexity.

A neural network model is defined by the following.

- An **architecture** defining the flow of information between computational layers. This defines the composition of functions that the network performs from input to output.
- A **cost function** (e.g. cross-entropy or mean squared error).
- An **optimization algorithm** (e.g. stochastic gradient descent with backpropagation).
- A set of **hyperparameters**. (Here we use this as a catch-all term to also include algorithm parameters that technically are not “hyperparameters” in the traditional sense because they don’t help you change the bias-variance tradeoff, such as the learning rate and the mini-batch size for stochastic gradient descent with mini-batches.)

Each *layer* is defined by the following components.

- A **parameterized function** that defines the layer’s map from input to output (e.g. $f(x) = \sigma(Wx + b)$).
- An **activation function** σ (e.g. ReLU, sigmoid, etc.).
- A set of **parameters** (e.g. weights and bias terms).

Neural networks are commonly used for supervised learning problems, where we have a set of inputs and a set of labels, and we want to learn the function that maps inputs to labels. To learn this function, we need to update the parameters of the network (i.e. the weights, including the bias terms). We do this using **mini-batch gradient descent**. To compute the gradients for gradient descent, we use a dynamic programming algorithm called **backpropagation**.

In the backpropagation algorithm, we first compute what is called a **forward pass** of the network. In the forward pass, we send a mini-batch of input data (e.g. 50 training points) through the network. The output is a set of predicted labels, which we use as input to our loss function (along with the true labels from the training data). We then take the gradients of the loss with respect to the parameters of each layer, starting with the output of the network and using the chain rule to propagate backwards through the layers. This is called the **backward pass**. During the backward pass we compute the gradients of the loss function with respect to each of the model parameters, starting from the last layer and “propagating” the information from the loss function backwards through the network. This lets us calculate gradients with respect to all the parameters of our network while letting us avoid computing the same gradients multiple times.

To summarize, training a neural network involves three steps.

1. Forward propagation of inputs.
2. Computing the cost.
3. Backpropagation and parameter updates.

2.2 Batching

When building neural networks, we have to carefully consider the data. In Homework 4, you coded both batch gradient descent and stochastic gradient descent for logistic regression. For the stochastic version, where only a single data point was used, the form of gradients used in gradient descent were different than those of batch gradient descent. Neural networks usually operate on mini-batches, or subsets of the data matrix. This is because iterating on all the data at once (batch gradient descent) is inefficient for large data sets, whereas iterating on just one training point at a time introduces excessive stochasticity (randomness) and makes poor use of your computer’s caches and potential for parallelism. Thus, every step of your neural network should be defined to operate on mini-batches of data. During a single operation of mini-batch gradient descent, you take a matrix of shape (B, d) where B is the mini-batch size and d is the number of features, and do a forward pass on B training points at once—ideally using vector operations to obtain some parallelism in your computations (as every training point is processed the same way). The input to a convolutional neural network for image recognition might be a four-dimensional array of shape (B, H, W, C) where B is the mini-batch size, H is the height of the image, W is the width of the image, and C is the number of channels in the image (3 for RGB—that is, red, green, and blue intensities).

As you are writing the gradient descent algorithm to work on mini-batches, all of your derivations must work for mini-batches. For that reason, many of the derivations you do for this homework will differ from those you have seen in class. Thinking in terms of mini-batches often changes the shapes and operations you do. **Your derivations must be for mini-batches and cannot use loops to iterate over individual data points.** Be prepared to spend some time working out the tricky details of how to do this.

2.3 Feed-Forward, Fully-Connected Neural Networks

A feed-forward, fully-connected neural network consists of layers of units alternating with layers of edges. Each layer of edges performs an **affine transformation** of an input, followed by a nonlinear activation func-

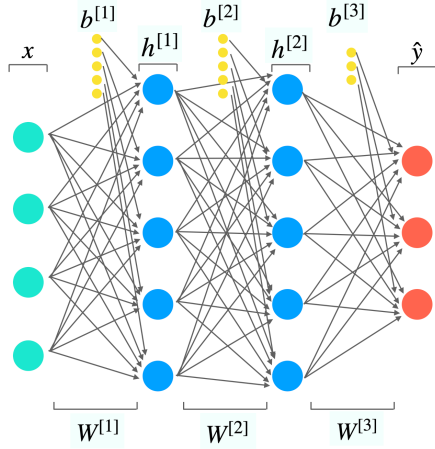


Figure 1: A 3-layer fully-connected neural network.

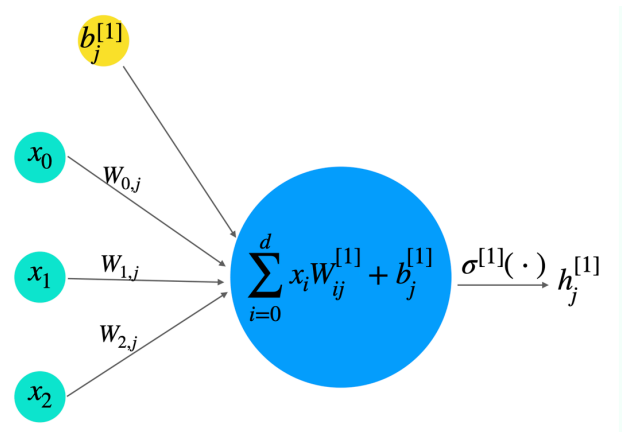


Figure 2: A single fully-connected neuron.

tion. “Fully-connected” means that a layer of edges connects every unit in one layer of units to every unit in the next layer of units. We use the following notation when defining fully-connected layers, with superscripts in brackets indexing layers (both layers of units and layers of edges) and subscripts indexing the vector/matrix elements. In this notation, we will use **row vectors** (not column vectors) to represent unit layers so that we can apply successive matrices (edge layers) to them from left to right.

- x : A single data vector, of shape $1 \times d$, where d is the number of features. You can think of it as “unit layer zero.” We present a training point or a test point here.
- $\hat{y}$: A single output vector, of shape $1 \times k$, where k is the number of output units. These could be regression values or they could symbolize classifications (and you can mix output units of both types). Each training point x is accompanied by a label vector y , and the goal of training is to make x ’s output $\hat{y}$ be close to y .
- $n^{[l]}$: The number of units (neurons) in unit layer l .
- $W^{[l]}$: A matrix of weights connecting unit layer $l-1$ with unit layer l , of shape $n^{[l-1]} \times n^{[l]}$. This matrix represents the weights of the connections in edge layer l . At edge layer 1, the shape is $d \times n^{[1]}$.
- $b^{[l]}$: The bias vector for layer l , of shape $1 \times n^{[l]}$.
- $h^{[l]}$: The output of edge layer l . This is a vector of shape $1 \times n^{[l]}$.
- $\sigma^{[l]}(\cdot)$: The nonlinear *activation function* applied at layer l .

A fully-connected layer l is a function

$$\phi^{[l]}(h^{[l-1]}) = \sigma^{[l]}(h^{[l-1]}W^{[l]} + b^{[l]}) = h^{[l]}.$$

The output $h^{[l]}$ of edge layer l is computed then used as the input to edge layer $l+1$. (At edge layer 1, $h^{[0]}$ is simply the input vector x .) A neural network is thus a composition of functions. We want to find weights such that the network maps each training point x to its label y .

In a multiclass classification problem with more than two classes, it is common to set k equal to the number of classes and have each output unit represent a true/false value for one class. This is called *one-hot encoding*. A one-hot encoded output unit used for classification might employ the labels

$$y_i = \begin{cases} 1 & x \in \text{class } i, \\ 0 & \text{otherwise.} \end{cases}$$

For example, for a classification problem with 3 classes, the label for a training point in class 3 might be $(0, 0, 1)$ and the label for a training point in class 2 might be $(0, 1, 0)$. However, the precise values you choose ought to depend on the activation function σ . Moreover, for reasons explained in lecture, you might get better results by using less extreme labels, such as 0.15 and 0.85, in lieu of 0 and 1, if σ is the logistic (sigmoid) function. If you have only two classes, there is usually no advantage to one-hot encoding; one output unit for the class label should suffice.

2.4 Convolutional Neural Networks

We will use the following notation when defining a convolutional neural network layer.

- X : A single image tensor (multi-dimensional array), of shape $d_1 \times d_2 \times c$, where d_1 and d_2 are the spatial dimensions, and c is the number of channels (of which there are typically three, encoding red, green, and blue pixel intensities).
- $\hat{y}$: A single output vector, of shape $1 \times k$.
- $n^{[l]}$: The number of image channels in layer l . Usually $n^{[l]} = c$.
- $(k_1, k_2)^{[l]}$: The size of the spatial dimensions of each filter/mask/kernel in layer l . Sometimes called the *kernel size*.¹
- $W^{[l]}$: The tensor of filters convolved at edge layer l . This tensor has shape $k_1 \times k_2 \times n^{[l-1]} \times n^{[l]}$.
- $b^{[l]}$: The bias vector for layer l , of shape $1 \times n^{[l]}$.
- $H^{[l]}$: The output of layer l . This is a tensor of shape $r_1 \times r_2 \times n^{[l]}$, where (r_1, r_2) is the shape of output of the convolution operation. Below we will discuss how to calculate this.
- $\sigma^{[l]}(\cdot)$: The nonlinear *activation function* applied at layer l .

In a convolutional layer, each filter is convolved with the input image, across every image channel. This operation is, essentially, a sliding sum of elementwise products. Figure 3 gives a visual example. Let Z denote the intermediate result *just before we apply the activation function* σ to obtain H . To compute a single element in the intermediate output Z , for a single unit n , we compute

$$Z[d_1, d_2, n] = (X * W)[d_1, d_2, n] = \sum_i \sum_j \sum_c W[i, j, c, n] X[d_1 + i, d_2 + j, c] + b[n].$$

Note: this formula is the **cross-correlation** formula from signal processing and NOT the convolution formula. Nevertheless this is what ML people call convolution, and so will we. It actually makes sense to use

¹“Kernel” is an overloaded word. In the context of convolutional networks, the convolutional filter is also called the “convolutional kernel.” This has no relationship with the “kernel” in the kernel trick and kernel methods. The convolutional kernel is also referred to as a “mask” in lecture.

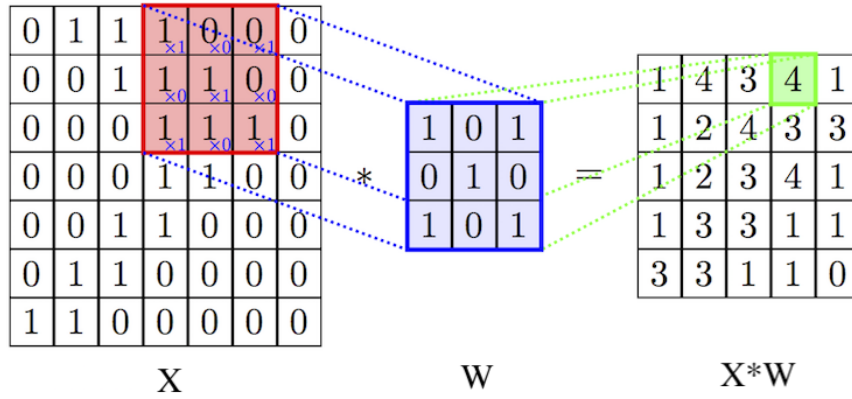


Figure 3: Figure showing an example of one convolution.

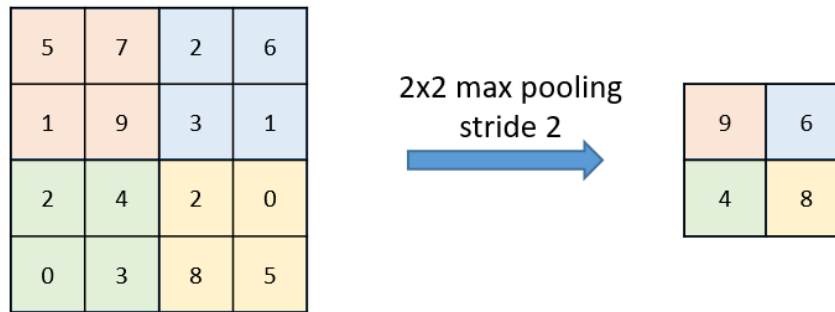


Figure 4: Figure showing an example of a max pooling layer with a kernel size of 2 and stride of 2.

cross-correlation instead of using convolution because the former can be interpreted as producing an output which is higher at locations where the image has the pattern in the filter and low elsewhere. Convolution is the same as cross-correlation with a flipped filter, and our filters are learned, so it makes no difference operationally whether you implement convolution or cross-correlation. However, to pass our tests, you must implement **cross-correlation** and call that convolution because that's how we do it in ML-land.

In this equation, we drop the layer superscripts for clarity, and index elements of the matrices in brackets. The pre-output Z is what we call a “feature map,” which essentially captures the strength of each filter at every region in the image. In the equation above, we slide the filter over the image in increments of one pixel. We can choose to take a larger steps instead. The size of the step taken in the convolution operation is referred to as the *stride*.

The output of the convolutional layer is

$$H^{[l]} = \sigma^{[l]}(Z^{[l]}).$$

A pooling layer is used to downsample the input feature maps. It takes an input array of shape $d_1 \times d_2 \times n$ and outputs an array of shape $r_1 \times r_2 \times n$. Note that it does **not** change the number of channels, but typically reduces the number of spatial dimensions, i.e., $r_1 < d_1$ and $r_2 < d_2$. In order to do this, we have a **kernel** of shape $k_1 \times k_2$ and a stride s . For each channel, we take either the max or the average of all the points in the window of size $k_1 \times k_2$. Then we slide the window by s pixels and repeat until we have performed this operation over the entire input image. This is illustrated in Figure 4. When the operation performed over each sliding window is max, it is called **max pooling**, whereas when the operation is averaging, then it is

called **average pooling**. Using similar notation as above, the function computed by a max pooling layer is

$$Z[r_1, r_2, c] = \text{MaxPool}(X)[r_1, r_2, c] = \max\{X[r_1s : r_1s + k_1 - 1, r_2s : r_2s + k_2 - 1, c]\}.$$

Replacing the max function with the average function, we get an average pooling layer. This function is abstracted away as `pool_fn` in the code. Note that `pool_fn` takes an array as input and outputs a single float. The notation on the right hand side should be read as array slicing as in `numpy`.

Traditional CNNs operate on images by combining convolutional layers with pooling layers to progressively “shrink” the spatial size of the input until it is small enough to be fed to fully-connected network layers for classification.

3 The Neural Nets Package

3.1 Structure

We have provided a modularized codebase for constructing neural networks. The codebase has the following structure.

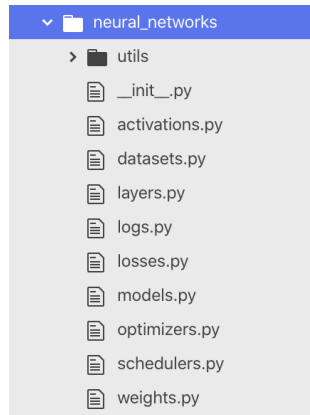


Figure 5: The structure of the starter codebase.

As you can see, the modules in the codebase reflect the structure outlined above. Different losses, activations, layers, optimizers, hyperparameters, and neural network architectures can be combined to yield different architectures.

In the codebase we have provided, each layer is an object with a few relevant attributes.

- **parameters**: An `OrderedDict` containing the weights and biases of the layer.
- **gradients**: An `OrderedDict` containing the gradients of the loss with respect to the weights and biases of the layer, with the same keys as **parameters**.
- **cache**: An `OrderedDict` containing intermediate quantities calculated in the forward pass that are useful for the backward pass.
- **activation**: An `Activation` instance that is the activation function applied by this layer.
- **n_in**: The number of input units (or, input channels in the case of a CNN).

- `n_out`: The number of output units (or, output channels in the case of CNN).

You will pass the layer a parameter that selects an activation function from those defined in `activations.py`. This will be stored as an attribute of the layer, which can be called as `layer.activation()`. The forward and backward passes of the layer are defined by the following methods.

- **forward** This method takes as input the output `X` from the previous layer (or input data). This method computes the function $\phi(\cdot)$ from above, combining the input with the weights `W` and bias `b` that are stored as attributes. It returns an output `out` and saves the intermediate value `Z` to the `cache` attribute, as it is needed to compute gradients in the backward pass.

```
def forward(self, X: np.ndarray) -> np.ndarray:
    """Forward pass: multiply by a weight matrix, add a bias, apply activation.
    Also, store all necessary intermediate results in the `cache` dictionary
    to be able to compute the backward pass.
    """
    ...
```

- **backward** This method takes the gradient of the downstream loss (i.e., the gradient of the loss with respect to the output of the current layer) as input. Then it uses the `cache` from the forward pass to compute the gradient of its output with respect to its inputs and weights. Via chain rule, it then returns the gradient of the loss with respect to the input of the layer.

```
def backward(self, dLdY: np.ndarray) -> np.ndarray:
    """Backward pass for fully connected layer.
    Compute the gradients of the loss with respect to:
    1. the weights of this layer (mutate the `gradients` dictionary)
    2. the bias of this layer (mutate the `gradients` dictionary)
    3. the input of this layer (return this)
    """
    ...
```

Each activation function has a similar (but simpler) structure:

```
class Linear(Activation):
    def __init__(self):
        super().__init__()

    def forward(self, Z: np.ndarray) -> np.ndarray:
        """Forward pass for f(z) = z."""
        return Z

    def backward(self, Z: np.ndarray, dY: np.ndarray) -> np.ndarray:
        """Backward pass for f(z) = z."""
        return dY
```

3.2 Testing

We have provided you with some local test cases that are run using the `unittest` module built into Python for testing. We test the `forward` and `backward` methods of all the layers that you implement, along with a few that you don't have to implement (which should already be passing their tests). We also test the weight initialization of the `FullyConnected` layer so that you have some guidance when you implement a layer for the first time. When you are done with the homework, **your code should be passing all the tests**, i.e. you should have 23/23 tests passing at the end.

To run all the tests, make sure that you are in the root directory of the project (and **not** in either of the `neural_networks/` and `tests/` directories). Then run the following:

```
python -m unittest -v
```

A total of 23 tests will be run. Before you implement anything, 15 tests should fail and 8 should pass.

The tests are located in the `tests/` directory. **Please do not modify the pre-existing tests** because they rely on data generated by a correct implementation, and any modifications can cause the tests to fail. The data is located in the `tests/data` directory. However, if you wish to write your own tests in addition to the ones that are already provided, then please refer to [this tutorial](#) and [the official documentation](#). Note that we don't require you to write your own tests as part of this homework.

The tests are organized into separate classes that mirror the structure of the classes that they test. For example, the `forward` method of the `Conv2D` class located in `neural_networks/layers.py` is tested by the `test_forward` method of the `TestConv2D` class located in `tests/test_layers.py`. If you want to run that specific test, you can use the following command **root directory of the project**:

```
python -m unittest -v tests.test_layers.TestConv2D.test_forward
```

If you instead want to run all the tests defined in the `TestConv2D` class, you should run:

```
python -m unittest -v tests.test_layers.TestConv2D
```

And if you want to run all the tests in `test_layers.py`, then you should run:

```
python -m unittest -v tests.test_layers
```

3.3 Autograder

Note that **the local tests above are not comprehensive, so it is possible that your code passes all the tests locally but is still broken and gives you poor performance**. They can still be a valuable debugging tool but if you want to properly test for correctness, submit your code to Gradescope by following the instructions on the front page of this PDF. The Gradescope autograder generates a suite of randomized tests (with randomized parameters and inputs) every time it is run and will also test for certain gradients that the local test cases miss.

You are welcome to submit to the autograder as frequently as you wish. However, since the local tests are quicker to run, are fully deterministic and don't require you to submit to Gradescope every time, they should be your first priority as a debugging tool.

3.4 Debugging

We have also included a python notebook named `check_gradients.ipynb`, which you can further use for debugging your layers' gradient computations. This can provide more debuggability than simply running the provided unit tests. It will compare the gradients you implement for various layers against gradients computed numerically as

$$\frac{\partial f}{\partial x_k}(a) \approx \frac{f(\dots, a_k + \epsilon, \dots) - f(\dots, a_k - \epsilon, \dots)}{2\epsilon}$$

for some small ϵ . If your gradient implementations are correct, they should be close enough to the numerical approximations such that the error between them is very small (usually on the order of 10^{-8} or smaller).

Something to note, however, is that this notebook only checks if the gradients you implement in the backward pass are consistent with the implementation of your forward pass. It's possible to have an incorrect

forward pass implementation and a consequently incorrect backward pass implementation that still yields low errors in the notebook. To check if your implementations are correct, you should submit to the Grade-scope autograder!

3.5 Submission

Please run

```
python3 generate_submission.py --help
```

for instructions on how to use the script to extract your implementations for the submission.

The script `generate_submission.py` extracts all the code from your implementations and produces either a $\text{\LaTeX}$ or markdown file containing your code implementations, when given the flag `--format latex` and `--format markdown` respectively. The generated document contains your functions in separate sections for activation functions, layers, losses, and the model. These can be sections, subsections, or subsubsections depending on whether you supply the flag `--heading_level 1`, `--heading_level 2`, or `--heading_level 3`.

For example, if you want $\text{\LaTeX}$ output with each part (activations, layers, losses, and the model) in a different subsection, with the output saved to `submission.tex`, you would run the following:

```
python3 generate_submission.py --format latex --heading_level 2 --output submission.tex
```

whereas if you want markdown output with each part (activations, layers, losses, and the model) in a different subsubsection, with the output saved to `submission.md`, you would run the following:

```
python3 generate_submission.py --format markdown --heading_level 3 --output submission.md
```

We would suggest running these commands to see exactly what they do.

The markdown document will compile by itself, but you would most likely want to create a markdown cell in a Jupyter Notebook and copy-paste the generated markdown into that cell. That should work seamlessly provided you can already compile Jupyter Notebooks into PDFs.

Note that the $\text{\LaTeX}$ document will **not** compile by itself. It is meant to generate code that you can then `\input{}` into your $\text{\LaTeX}$ document.

Feel free to play around with the script if you want to. Notably, if you change some function which is not in the `student_implementations` list, then you could add that function to the `student_implementations` list to have the script automatically gather your code from that function (but we think that most students will **not** have to do this).

Questions

The background section of the homework has ended. The following sections contain the questions you must complete.

4 Basic Network Layers

In this question you will implement the layers needed for basic classification neural networks. For each part, you will be asked to 1) derive the gradients, 2) write the matching code, 3) pass the tests.

Keep in mind that your solutions to all layers **must operate on mini-batches of data** and should not use loops to iterate over the training points in a mini-batch.

4.1 ReLU

First, you will implement the ReLU activation function in `activations.py`. ReLU is a very common activation function that is typically used in the hidden layers of a neural network and is defined as

$$\sigma_{\text{ReLU}}(\gamma) = \begin{cases} 0 & \gamma < 0, \\ \gamma & \text{otherwise.} \end{cases}$$

Note that the activation function is applied elementwise to a vector or matrix input.

Instructions

1. First, derive the gradient of the downstream loss with respect to the input of the ReLU activation function, Z . **You must arrive at a solution in terms of $\partial L / \partial Y$, the gradient of the loss w.r.t. the output of ReLU $Y = \sigma_{\text{ReLU}}(Z)$, and the batched input Z , i.e., where $Z \in \mathbb{R}^{m \times n}$.** Include your derivation in your writeup.
Hint: you are allowed to use operations like elementwise multiplication and/or division!
2. Next, implement the forward and backward passes of the ReLU activation in the script `activations.py`. Include your code in your writeup (either a screenshot or typesetting is fine). **Do not iterate over training examples, use batched operations.**

$$1. \quad Y = \sigma_{\text{ReLU}}(Z) \quad Z \in \mathbb{R}^{m \times n} \Rightarrow Y \in \mathbb{R}^{m \times n}$$

$$\frac{\partial Y_{ij}}{\partial Z_{ij}} = \begin{cases} 1 & \text{if } Z_{ij} > 0 \\ 0 & \text{if } Z_{ij} \leq 0 \end{cases}$$

$$\left[\frac{\partial L}{\partial Z} \right]_{ij} = \frac{\partial L}{\partial Z_{ij}} = \frac{\partial L}{\partial Y_{ij}} \cdot \frac{\partial Y_{ij}}{\partial Z_{ij}} = \frac{\partial L}{\partial Y_{ij}} \mathbb{1}_{Z_{ij} > 0}$$

$$\Rightarrow \frac{\partial L}{\partial Z} = \frac{\partial L}{\partial Y} \odot \mathbb{1}_Z > 0$$

4.2 Fully-Connected Layer

Now you will implement the forward and backward passes for the fully-connected layer in the `layers.py` script. The code is marked with `YOUR CODE HERE` statements indicating what to implement and where. Please read the docstrings and the function signatures too. Write the fully-connected layer for a general input h that contains a mini-batch of m examples with d features.

When implementing a new layer, it is important to manually verify correctness of the forward and backward passes. For debugging tips, please refer back to section 3.3.

Instructions

1. Derive the gradients of the loss $L \in \mathbb{R}$ with respect to weight matrix $W \in \mathbb{R}^{n^{[l]} \times n^{[l+1]}}$ and bias *row* vector $b \in \mathbb{R}^{1 \times n^{[l+1]}}$ in the fully-connected layer, $\partial L / \partial W$ and $\partial L / \partial b$. You will also need to take the gradient of the loss with respect to the input of the layer $\partial L / \partial X$, which will be passed to lower layers, where $X \in \mathbb{R}^{m \times n^{[l]}}$ is the batched input. **Again, you must arrive at a solution that uses batched X and Z . Please express your solution in terms of $\partial L / \partial Z$, which you have already obtained in 4.1, where $Z = XW + 1b$ and $1 \in \mathbb{R}^{m \times 1}$ is a column of ones.** Note that $1b$ is a matrix (it's an outer product) whose each row is the row vector b so we are adding the same bias vector to each sample during the forward pass: this is the mathematical equivalent of numpy broadcasting. Include your derivations in your writeup.
2. Implement the forward and backward passes of the fully-connected layer in `layers.py`. First, initialize the weights of the model using `_init_parameters`, which takes the shape of the design matrix as input and initializes the parameters, cache, and gradients of the layer. The backward method takes in an argument `dLdY`, the gradient of the loss with respect to the output of the layer, which is computed by higher layers and backpropagated. This should be incorporated into your gradient calculation. In your writeup, include the code you have implemented. **Do not loop over training points; use batched operations.**

$$1. \quad Z = XW + 1b$$

$$Y = \sigma(Z)$$

$$\text{shape : } \quad X : m \times n^{[l]} \quad W : n^{[l]} \times n^{[l+1]} \\ b : 1 \times n^{[l+1]} \quad Z : m \times n^{[l+1]}$$

$$Z_{ij} = \sum_k X_{ik} W_{kj} + b_j$$

$$\Rightarrow \frac{\partial Z_{mn}}{\partial X_{ij}} = \frac{\partial \sum_k X_{mk} W_{kn}}{\partial X_{ij}} = \delta_{mi} \cdot W_{jn}$$

$$\Rightarrow \left[\frac{\partial L}{\partial X} \right]_{ij} = \frac{\partial L}{\partial X_{ij}} = \sum_{m,n} \frac{\partial L}{\partial Z_{mn}} \frac{\partial Z_{mn}}{\partial X_{ij}} = \sum_n \frac{\partial L}{\partial Z_{in}} \frac{\partial Z_{in}}{\partial X_{ij}}$$

$$= \sum_n \frac{\partial L}{\partial Z_{in}} \cdot W_{jn} = \sum_n \left[\frac{\partial L}{\partial Z} \right]_{in} W_{nj}^T = \left[\frac{\partial L}{\partial Z} W^T \right]_{ij}$$

$$\Rightarrow \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} W^T$$

$$\frac{\partial z_{mn}}{\partial w_{ij}} = \frac{\partial \sum_k x_{mk} w_{kn}}{\partial w_{ij}} = x_{mi} \cdot \delta_{jn}$$

$$\begin{aligned} \left[\frac{\partial L}{\partial w} \right]_{ij} &= \frac{\partial L}{\partial w_{ij}} = \sum_{m,n} \frac{\partial L}{\partial z_{mn}} \frac{\partial z_{mn}}{\partial w_{ij}} = \sum_{m,n} \frac{\partial L}{\partial z_{mn}} \cdot x_{mi} \cdot \delta_{jn} \\ &= \sum_m \frac{\partial L}{\partial z_{mj}} \cdot x_{mi} = \sum_m x_{im}^T \cdot \frac{\partial L}{\partial z_{mj}} = \left[x^T \frac{\partial L}{\partial z} \right]_{ij} \end{aligned}$$

$$\Rightarrow \frac{\partial L}{\partial w} = x^T \frac{\partial L}{\partial z}$$

$$\frac{\partial z_{mn}}{\partial b_{ij}} = 1 \cdot \delta_{jn}$$

$$\left[\frac{\partial L}{\partial b} \right]_{ij} = \frac{\partial L}{\partial b_{ij}} = \sum_{m,n} \frac{\partial L}{\partial z_{mn}} \cdot \frac{\partial z_{mn}}{\partial b_{ij}} = \sum_m \frac{\partial L}{\partial z_{mj}} \cdot 1 = \left[1^T \frac{\partial L}{\partial z} \right]_{ij}$$

$$\Rightarrow \frac{\partial L}{\partial b} = 1^T \frac{\partial L}{\partial z}$$

4.3 Softmax Activation

Next, we need to define an activation function for the output layer. The ReLU activation function returns continuous values that are (potentially) unbounded to the right. Since we are building a classifier, we want to return *probabilities* over classes. The softmax function has the desirable property that it outputs a probability distribution. That is, the softmax function squashes continuous values into the range $[0, 1]$ and normalizes the outputs so that they add up to 1. For this reason, many classification neural networks use the softmax activation. The softmax activation takes in a vector s of k un-normalized values $s_1, \dots, s_k$ and outputs a probability distribution over the k possible classes. The forward pass of the softmax activation on input s_i is

$$\sigma_i = \frac{e^{s_i}}{\sum_{j=1}^k e^{s_j}},$$

where k ranges over all elements in s . Due to issues of numerical stability, the following modified version of this function is commonly used.

$$\sigma_i = \frac{e^{s_i - m}}{\sum_{j=1}^k e^{s_j - m}},$$

where $m = \max_{j=1}^k s_j$. We strongly recommend implementing the latter formula.

Instructions

1. Derive the Jacobian of the softmax activation function. **You do not need to use batched inputs for this question; an answer for a single training point is acceptable. You do not need to write out the entire matrix, but please write out what $\partial \sigma_i / \partial s_j$ is for an arbitrary (i, j) pair.** Include your derivation in your writeup.
2. Implement the forward and backward passes of the softmax activation in `activations.py`. We recommend vectorizing the backward pass for efficiency. **For this question only may you use a “for” loop over the training points in the mini-batch.** In your writeup, include the code you have implemented.

1. The Jacobian of $\sigma(s)$ is

$$\frac{\partial \sigma}{\partial s} = \begin{bmatrix} \frac{\partial \sigma_1}{\partial s_1} & \frac{\partial \sigma_1}{\partial s_2} & \dots & \frac{\partial \sigma_1}{\partial s_k} \\ \frac{\partial \sigma_2}{\partial s_1} & \frac{\partial \sigma_2}{\partial s_2} & & \vdots \\ \vdots & & \ddots & \\ \frac{\partial \sigma_k}{\partial s_1} & \dots & \dots & \frac{\partial \sigma_k}{\partial s_k} \end{bmatrix}$$

$$\left[\frac{\partial \sigma}{\partial s} \right]_{ij} = \frac{\partial \sigma_i}{\partial s_j} = \frac{\partial}{\partial s_j} \frac{e^{s_i}}{\sum_{j=1}^k e^{s_j}}$$

① when $i=j$

$$\frac{\partial \sigma_i}{\partial s_j} = \frac{\partial}{\partial s_i} \frac{e^{s_i}}{\sum_{i=1}^k e^{s_i}}$$

$$= \frac{e^{s_i}}{\sum_{i=1}^k e^{s_i}} + e^{s_i} \frac{-1}{\left(\sum_{i=1}^k e^{s_i}\right)^2} e^{s_i}$$

$$= \sigma_i - \sigma_i^2$$

② when $i \neq j$

$$\begin{aligned} \frac{\partial \sigma_i}{\partial s_j} &= e^{s_i} \frac{\partial}{\partial s_j} \left[\sum_{j=1}^k e^{s_j} \right]^{-1} \\ &= e^{s_i} \frac{-1}{\left(\sum_{j=1}^k e^{s_j}\right)^2} \cdot e^{s_j} \\ &= - \frac{e^{s_i}}{\sum_{j=1}^k e^{s_j}} \cdot \frac{e^{s_j}}{\sum_{j=1}^k e^{s_j}} = - \sigma_i \cdot \sigma_j \end{aligned}$$

$$\text{①②} \Rightarrow \left[\frac{\partial \sigma}{\partial s} \right]_{ij} = \begin{cases} \sigma_i(1 - \sigma_i), & i=j \\ -\sigma_i \sigma_j, & i \neq j \end{cases}$$

4.4 Cross-Entropy Loss

For this classification network, we will be using the multi-class cross-entropy loss function

$$L = -y \cdot \ln \hat{y},$$

where y is the binary one-hot vector encoding the ground truth labels and $\hat{y}$ is the network's output, a vector of probabilities over classes. Note that $\ln \hat{y}$ is $\hat{y}$ with the natural log applied elementwise to it and $\cdot$ represents the dot product between y and $\ln \hat{y}$. The cross-entropy cost calculated for a mini-batch of m samples is

$$L = -\frac{1}{m} \left(\sum_{i=1}^m y_i \cdot \ln(\hat{y}_i) \right).$$

Let $Y \in \mathbb{R}^{m \times k}$ and $\hat{Y} \in \mathbb{R}^{m \times k}$ be the one-hot labels and network outputs for the m samples, stacked in a matrix. Then, y_i and $\hat{y}_i$ in the expression above are just the i th rows of Y and $\hat{Y}$.

Instructions

1. Derive $\partial L / \partial \hat{Y}$ the gradient of the cross-entropy cost with respect to the network's predictions, $\hat{Y}$. **You must use batched inputs.** Include your derivation in your writeup.

Hint: you are allowed to use operations like elementwise multiplication and/or division!

2. Implement the forward and backward passes of the cross-entropy cost. Note that in the codebase we have provided, we use the words “loss” and “cost” interchangeably. This is consistent with most large neural network libraries, though technically “loss” denotes the function computed for a single datapoint whereas “cost” is computed for a batch. You will be computing over batches. **Do not iterate over training examples; use batched operations.** In your writeup, include the code you have implemented.

$$\begin{aligned} 1. \left[\frac{\partial L}{\partial \hat{Y}} \right]_{ij} &= \frac{\partial L}{\partial \hat{y}_{ij}} = \frac{\partial}{\partial \hat{y}_{ij}} \left(-\frac{1}{m} \sum_{i=1}^m y_i \cdot \ln \hat{y}_i \right) \\ &= -\frac{1}{m} \frac{\partial}{\partial \hat{y}_{ij}} \sum_{i=1}^m \sum_{j=1}^k y_{ij} \cdot \ln \hat{y}_{ij} \\ &= -\frac{1}{m} \frac{(y_i)_j}{(\hat{y}_i)_j} = -\frac{1}{m} \frac{y_{ij}}{\hat{y}_{ij}} \end{aligned}$$

$$\Rightarrow \frac{\partial L}{\partial \hat{Y}} = -\frac{1}{m} \frac{Y}{\hat{Y}} \quad \text{where } \frac{Y}{\hat{Y}} \text{ represents elementwise division.}$$

5 Two-Layer Fully Connected Networks

Now, you will use the methods you've written to train a two-layer network (also referred to as a one-*hidden*-layer network). You will use the **Iris Dataset**, which contains 4 features for 3 different classes of irises.

Instructions

1. Fill in the `forward`, `backward`, and `predict` methods for the `NeuralNetwork` class in `models.py`. In your writeup, include the code you have implemented. Define the parameters of your network in `train_ffnn.py`. We have provided you with several other classes that are critical for the training process.
 - The data loader (in `datasets.py`), which is responsible for loading batches of data that will be fed to your model during training. You may wish to alter the data loader to handle data pre-processing. Note that all datasets you are given have not been normalized or standardized.
 - The stochastic gradient descent optimizer (in `optimizers.py`), which performs the gradient updates and optionally incorporates [a momentum term](#).
 - The learning rate scheduler (in `schedulers.py`), which handles the optional learning rate decay. You may choose to use either a constant or exponentially decaying learning rate.
 - Weight initializers (in `weights.py`). We provide you with many options to explore, but we recommend using `xavier_uniform` as a default.
 - A logger (in `logs.py`), which saves hyperparameters and learned parameters and plots the loss as your model trains.

Outputs will be saved to the folder `experiments/`. You can change the name of the folder a given run saves to by changing the parameter called `model_name`. Be careful about overwriting folders; if you forget to change the name and perform a run with identical hyperparameters, your previous run will be overwritten!

2. Train a 2-layer neural network on the Iris Dataset while varying the following hyperparameters.
 - Learning rate
 - Hidden layer size (number of units per hidden layer)

You must try at least 3 different *combinations* of these hyperparameters. Report the results of your exploration, including the values of the parameters you explored and which set of parameters gave the best test error. Provide plots showing the loss versus iterations for your best model and report your final test error.

6 CNN Layers

In this problem, you will write the forward and backward passes for the convolutional and pooling layers that are used within convolutional neural networks.

Again, remember that all your implementations and derived gradients must be for mini-batches, and not single examples. For both CNN layers, the input tensor will be of shape (m, H, W, C) where m denotes the number of mini-batch samples, H and W are height and width of each sample (the spatial dimensions) and C is the number of channels (3 for RGB images of 1 for grayscale images).

6.1 The Einsum Function

The `np.einsum` function is a powerful tool for performing various linear algebra operations on arrays in NumPy, a popular numerical computing library in Python. It allows for efficient computation of multi-dimensional array operations through a concise and flexible notation.

The `np.einsum` function takes in one or more input arrays and a string specifying the desired output format. The string consists of subscripts that define how the arrays should be multiplied, summed, and contracted. The output of the function is a new array that represents the result of the specified operation.

Here are some basic rules that help you understand how to use `np.einsum`, as this may help you with some of the coding parts in the CNN section.

- Einsum notation uses subscripts to represent the dimensions of the input arrays. Each subscript string is a range of single uppercase or lowercase letters, with each letter denoting an axis of a numpy array. For example, the subscript string `i` represents a 1-D array of length i while the subscript string `ijk` represents an array of shape (i, j, k) .
- The einsum string consists of two or more subscript strings separated by a comma, followed by an arrow `->`, and a final subscript string that specifies the dimensions of the output array. For example, `i, j->ij` specifies the multiplication of two arrays with of lengths i and j , resulting in an output array with shape (i, j) . You might recognize this as the outer product between the two input arrays.
- The same subscript letter can appear in multiple input subscript strings, indicating that the corresponding dimensions should be multiplied and summed over. For example, `ij, j->i` performs an elementwise multiplication and sum over the second axis of the first input array and the entire second input array, resulting in an output array with one axis and length i . You might recognize this as the regular matrix-vector product between the first input array (a matrix of shape (i, j)) and the second input array (a vector of length j).
- If a subscript appears only in the output subscript string, the corresponding axis is preserved in the output array. For example, `ij->j` returns a one-dimensional array of length j , whose components are the components of the input array summed along the first axis.
- Ellipses (...) can be used to represent a range of axes in an input or output subscript string. For example, `i...->i` performs a summation over all axes except the first of the input array, resulting in a one-dimensional output array with length i .

For more details please refer to the NumPy API. This [tutorial](#) should also be very helpful!

Please accomplish the following tasks using both regular NumPy methods and `np.einsum`. Confirm you get the same answer by subtracting the two results and show that its norm is zero (or virtually zero, barring

numerical approximation errors). In your writeup, include any code you write as well as a confirmation that both methods yield the same result (for example, printing the norm and including it as a screenshot is fine).

1. For a random matrix $A \in \mathbb{R}^{5 \times 5}$, find its trace.
2. For random matrices $A, B \in \mathbb{R}^{5 \times 5}$, compute their matrix product.
3. For a batch of random matrices of shapes (3, 4, 5) and (3, 5, 6) (the batch size is 3 here), compute their batchwise matrix product (the resulting batch will have shape (3, 4, 6)).

1. `np.einsum("ii->", A)`
`np.trace(A)`
2. `np.einsum("ik,kj->ij", A, B)`
`np.dot(A, B)`
3. `np.einsum("ijl,ilk->ijk", A, B)`
`np.matmul(A, B)`

6.2 Convolutional Layer

- Derive the gradient of the loss with respect to the input and parameters (weights and biases) of a convolutional layer. **For this question your answer may be in the form of individual component partial derivatives. Assume you have access to the full 3d array $\frac{\partial L}{\partial Z[d_1, d_2, n]}$, which is the gradient of the pre-activation w.r.t the loss** You may also ignore stride and assume both the filter and image are infinitely zero-padded outside of their bounds. Include your derivations in your writeup.

- What is $\frac{\partial L}{\partial b[f]}$ for an arbitrary $f \in [1, \dots, n]$?
- What is $\frac{\partial L}{\partial W[i, k, c, f]}$ for arbitrary i, k, c, f indexes?
- What is $\frac{\partial L}{\partial X[x, y, c]}$ for arbitrary x, y, c indexes?

- Fill in the forward and backward passes of the Conv2D layer in `layers.py`. In your writeup, include the code you have implemented. Unlike the mathematical derivation, you will have to account for padding and stride, as well as batching, in your code. **Do not iterate over training examples; use batched operations.** You may find the einsum operation helpful here.

$$1. \quad Z[d_1, d_2, n] = (X * W)[d_1, d_2, n] = \sum_i \sum_k \sum_c W[i, k, c, n] X[d_1 + i, d_2 + k, c] + b[n]$$

$$(a) \quad \frac{\partial Z[d_1, d_2, n]}{\partial b[f]} \overset{\text{Convolution}}{=} \delta_{nf}$$

$$\Rightarrow \frac{\partial L}{\partial b[f]} = \sum_{d_1} \sum_{d_2} \sum_n \frac{\partial L}{\partial Z[d_1, d_2, n]} \cdot \frac{\partial Z[d_1, d_2, n]}{\partial b[f]} \\ = \sum_{d_1} \sum_{d_2} \sum_n \frac{\partial L}{\partial Z[d_1, d_2, n]} \cdot \delta_{nf} = \sum_{d_1} \sum_{d_2} \frac{\partial L}{\partial Z[d_1, d_2, f]}$$

$$(b) \quad \frac{\partial Z[d_1, d_2, n]}{\partial W[i, k, c, f]} = \delta_{nf} \cdot X[d_1 + i, d_2 + k, c] \\ \Rightarrow \frac{\partial L}{\partial W[i, k, c, f]} = \sum_{d_1} \sum_{d_2} \sum_n \frac{\partial L}{\partial Z[d_1, d_2, n]} \cdot \frac{\partial Z[d_1, d_2, n]}{\partial W[i, k, c, f]} \\ = \sum_{d_1} \sum_{d_2} \frac{\partial L}{\partial Z[d_1, d_2, f]} \cdot X[d_1 + i, d_2 + k, c]$$

$$(c) \quad \frac{\partial Z[d_1, d_2, n]}{\partial X[x, y, c]} = W[x - d_1, y - d_2, c, n]$$

$$\nwarrow x = d_1 + i, \quad y = d_2 + k \Rightarrow W[i, k, c, n] = W[x - d_1, y - d_2, c, n]$$

$$\Rightarrow \frac{\partial L}{\partial X[x, y, c]} = \sum_{d_1} \sum_{d_2} \sum_n \frac{\partial L}{\partial Z[d_1, d_2, n]} \cdot \frac{\partial Z[d_1, d_2, n]}{\partial X[x, y, c]} \\ = \sum_{d_1} \sum_{d_2} \sum_n \frac{\partial L}{\partial Z[d_1, d_2, n]} \cdot W[x - d_1, y - d_2, c, n]$$

6.3 Pooling Layers

1. Explain how we can use the backprop algorithm to compute gradients through the max pooling and average pooling operations. (A plain English answer will suffice; equations are optional.) Carefully consider the values you must keep track of!
2. Fill in the forward and backward passes of the Pool2D layer in `layers.py`. In your writeup, include the code you have implemented. **Do not iterate over training examples, use batched operations.**

6.4 (Optional) Convolutional Neural Network

Convolutional neural networks take considerably longer to train than the feedforward layers we have built, and the numpy implementation you have written will be impractically unoptimized compared to industry-grade neural network packages. So, you are not required to train a proper CNN for this part of the assignment.

That said, if you wish to see your convolutional and pooling layers in action, we have included a file called `train_conv.py` where you can construct a CNN and train it on the **MNIST Dataset**. This can be another good way to test the correctness of your CNN layers (think of this as an integration test compared to all the unit tests). The default architecture implemented in that file is called **LeNet** and, along with the default hyperparameters, it should achieve around a 96 – 97% validation accuracy after just one training epoch, assuming your neural network layers are correctly implemented. Feel free to play around with different hyperparameters to see how they might impact CNN training. You don't have to submit anything for this part in your writeup.

6.3

1. For max pooling, the gradient from the next layer is passed back only to the neuron which achieved the max. All other neurons get zero gradient. Because max pooling does not have any trainable parameters, we won't need to worry about calculating any gradients for weights. Once we have the gradient with respect to the input, the backprop algorithm can continue on to the layer before the maxpool by using this gradient.

For a pooling region P where $Z = \max(P, \text{keepdims} = \text{True})$

$$\frac{\partial L}{\partial X[i,j]} = \begin{cases} \frac{\partial L}{\partial Z[i,j]} & \text{if } X[i,j] = Z[i,j] = \max(P) \\ 0 & \text{otherwise} \end{cases}$$

Average pooling is analogous except with the average operation instead of the max.

$$Z = \text{Average}(P) \Rightarrow \frac{\partial}{\partial X[i,j]} = \frac{1}{k_h \times k_w} \frac{\partial}{\partial Z[i,j]}$$

$P: k_h \times k_w$

7 PyTorch

Please see the Google Colab Notebook [here](#) that contains the questions.

As with every homework, you are allowed to use any setup you wish. However, we highly recommend that you use [Google Colab](#) for free access to GPUs, which will significantly improve the speed of neural network training. Instructions are provided in the notebook itself. If you have access to GPUs locally, then feel free to run the notebook on your computer.

The following sections mirror the Colab Notebook and provide the deliverables for each question.

7.1 MLP for Fashion MNIST

Deliverables:

1. Provide code for training an MLP on the fashion MNIST dataset (can be tagged from colab notebook and in the code appendix)
2. A plot of the training and validation loss for each epoch of training for at least 8 epochs.
3. A plot of the training and validation accuracy for each epoch, achieving a final validation accuracy of at least 82%.

7.2 CNNs for CIFAR-10

Deliverables:

1. Provide the code for training your CNN model (can be in appendix).
2. Submit to Kaggle and include your test accuracy in your report. Our simple reference solution gets a test accuracy of 74.8% after 10 epochs.
3. Provide at least 1 training curve for your model, depicting loss per epoch or step after training for at least 8 epochs.
4. Explain the components of your final model, and how you think your design choices contributed to its performance.

Submission Checklist

Please ensure you have completed the following before your final submission.

At the beginning of your writeup...

1. Have you copied and hand-signed the honor code specified in Question 1?
2. Have you listed all students (Names and ID numbers) that you collaborated with?

In your writeup for Question 7...

1. Have you included your **Kaggle Score** and **Kaggle Username**?
2. Have you included your generated plots and visualizations?

At the end of the writeup...

1. Have you provided a code appendix including all code you wrote in solving the homework?

Code Submission

1. Have you dragged and dropped every file in the [neural networks package](#), along with `train_ffnn.py`, into the Gradescope submission box?
2. Have you included a `README.md` in your archive containing any special instructions to reproduce your results?
3. Have you included the code you wrote for the PyTorch section with the rest of your submission?

Submissions

1. Have you submitted your written solutions to the Gradescope assignment titled **HW6 Write-Up** and selected pages appropriately?
2. Have you submitted your code files to the Gradescope assignment titled **HW6 Code**?
3. Have you submitted your test set predictions for **CIFAR-10** dataset to the appropriate Kaggle challenge?
4. Is your Kaggle submission in integer format? Submissions in decimal format will receive a score of zero!

Congratulations! You have completed Homework 6.